

Todo list

- Maybe go back and look at Flux Density and how i think need to be percendicalur
to someting 4

Sensors & Systems

Simple Sensors — Slides 1–9

Jan Bendtsen, Torben Knudsen | Electronic Systems, 8th Semester

Slides 1–3 What Is a “Simple” Sensor?

The lecture covers: Hall sensor, temperature sensor, humidity sensor, measurement noise (Gaussian, thermal, flicker, shot), and simple filtering.

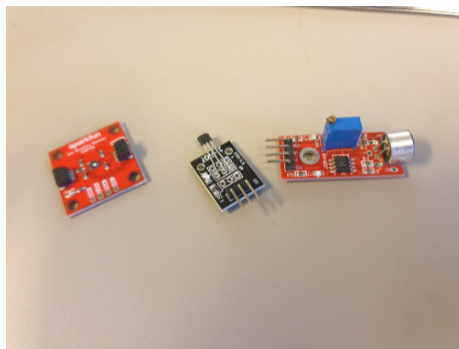


Figure 1: Three “simple” sensors: air-quality (left), Hall effect (centre), microphone (right).

A **simple sensor** is a self-contained device that converts a single, continuous physical quantity into an electrical signal with little or no additional computation required on the user’s side. Three defining properties hold for all simple sensors:

- It measures **one continuous physical quantity** — temperature, magnetic field strength, sound pressure, humidity, etc.
- It provides a **simple interface** — either an analog voltage/current output or a straightforward digital protocol (I²C, SPI, UART, etc.).
- It does **not require extensive processing** to obtain the desired measurement — contrast this with a camera or radar, where significant computation must precede any useful output.

Definition — Simple Sensor: a sensor that (1) measures a single continuous physical quantity, (2) provides a simple analog or digital interface, and (3) requires no extensive post-processing to obtain the desired information.

Slide 4 The Sensor Signal Chain

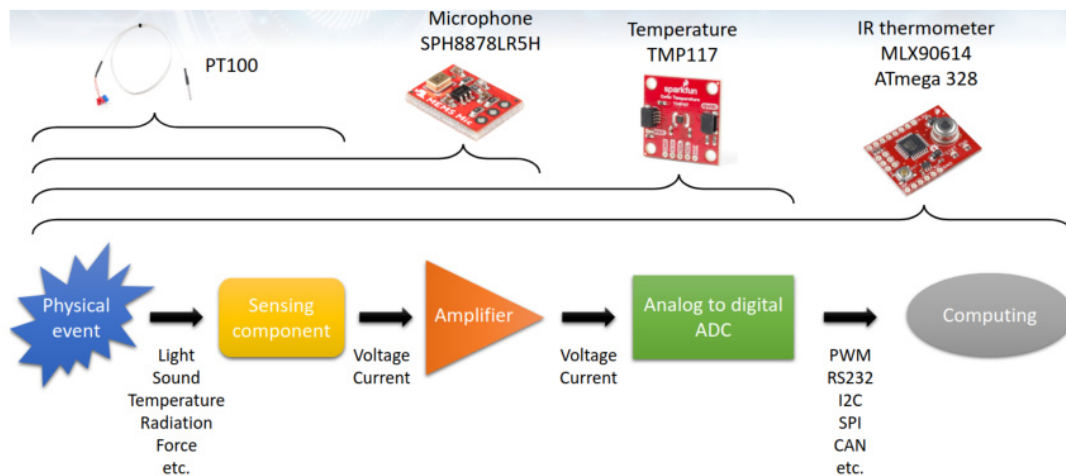


Figure 2: The general sensor signal chain: physical event $\rightarrow$ sensing component $\rightarrow$ amplifier $\rightarrow$ ADC $\rightarrow$ computing.

Every sensor — simple or otherwise — follows the same general processing pipeline from the physical world to a number in your microcontroller. Understanding this chain is important because it tells you *where* a particular sensor sits and what processing remains your responsibility.

1. **Physical event.** A real-world phenomenon occurs — light, sound, temperature change, force, radiation. This is the stimulus the sensor exists to measure.
2. **Sensing component.** A transducer converts the physical phenomenon into an electrical signal, typically a voltage or current. This is the core physics step. The output is often tiny — millivolts or microamps.
3. **Amplifier.** The weak electrical signal is boosted to a level suitable for the ADC. Because sensor signals are small, amplifier quality matters; any noise or offset introduced here gets amplified alongside the signal.
4. **Analog-to-Digital Converter (ADC).** The amplified continuous signal is sampled at regular intervals T_s and converted to a digital integer. This is where the continuous physical world becomes a discrete-time sequence of values. Output protocols include PWM, RS232, I²C, SPI, and CAN.
5. **Computing.** A microcontroller or processor interprets the stream of digital values and acts on them.

Note — integration level. Different sensors integrate different numbers of these stages internally. A bare PT100 resistance thermometer gives you only a raw resistance

— stages 2 onward are your problem. A TMP117 integrates the amplifier and ADC on-chip and hands you temperature in °C directly over I²C. Always check the datasheet to know what the sensor does for you.

Key Takeaway: The same five-stage chain governs every sensor system. “Simple” sensors merely make stages 3–4 (and sometimes 5) invisible to the user by integrating them on-chip.

Slides 5–7 The Hall Effect Sensor

What Is a Hall Sensor For?

A Hall sensor is used to **measure or detect magnetic fields**. Just as a thermistor converts temperature into a resistance, a Hall sensor converts magnetic field strength (flux density, measured in Gauss or Tesla) into a voltage.

Practical uses include:

- Detecting the presence of a magnet (e.g. reed-switch replacement, door/lid open–closed detection).
- Measuring rotation speed — a magnet on a shaft passes the sensor once per revolution, producing a pulse.
- Measuring current — a current-carrying conductor generates a magnetic field proportional to the current.
- Measuring the position of a magnetic actuator or joystick.

The Physics of the Hall Effect (Slide 6)

Maybe go back and look at Flux Density and how i think need to be percendicalur to someting

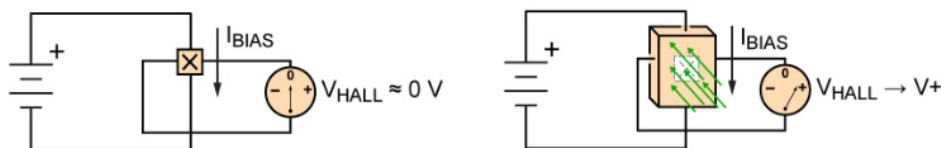


Figure 3: Left: no magnetic field, $V_{HALL} \approx 0 \text{ V}$. Right: magnetic field perpendicular to bias current, V_{HALL} rises proportionally.

Inside the sensor there is a thin **semiconductor sheet**. A constant voltage source forces a fixed **bias current** I_{BIAS} through the sheet along its length.

- **No field:** charge carriers flow straight through the sheet; the voltage measured *across the width*, V_{HALL} , is approximately zero.
- **Field present:** flux lines enter the semiconductor perpendicular to I_{BIAS} . The Lorentz force $\mathbf{F} = q\mathbf{v} \times \mathbf{B}$ deflects the charge carriers sideways, causing charge accumulation along one edge. This creates a measurable transverse voltage — V_{HALL} rises toward a positive value.

The key relationship is:

$$V_{\text{HALL}} \propto I_{\text{BIAS}} \cdot B$$

where B is the magnetic flux density. Since I_{BIAS} is kept constant, V_{HALL} varies **linearly** with B .

Variable summary:

- I_{BIAS} — constant bias current forced through the semiconductor sheet.
- B — magnetic flux density (Gauss / Tesla); the quantity being measured.
- V_{HALL} — output voltage across the *width* of the sheet; ≈ 0 with no field, proportional to B when a field is present.

Signal Conditioning — Two Configurations (Slide 7)

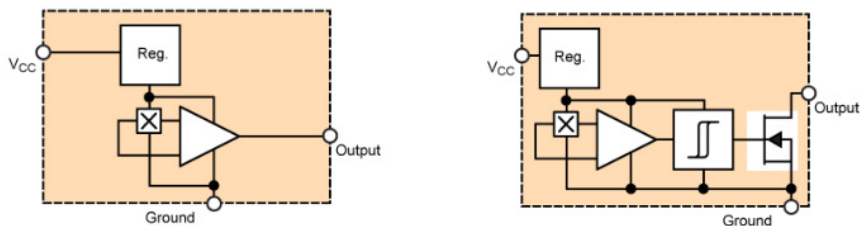


Figure 4: Left: linear Hall sensor (Hall element + OpAmp only). Right: switch-type Hall sensor (Hall element + OpAmp + comparator/Schmitt trigger).

V_{HALL} is typically only a few millivolts — far too small to be useful on its own. Two different goals lead to two different circuit configurations:

Configuration 1 — Measuring Field Strength (linear sensor)

Connect a precision OpAmp directly after the Hall element. The OpAmp amplifies V_{HALL} to a usable level; the output is an analog voltage proportional to B .

Warning: this configuration demands a **high-quality, low-offset OpAmp**. Any DC offset is amplified alongside the signal — a 1 mV offset with a gain of 500 becomes 500 mV of error, completely swamping a small magnetic signal.

Configuration 2 — Detecting Presence Only (switch-type sensor)

Add a **comparator (Schmitt trigger)** after the amplifier. The comparator outputs a clean digital HIGH or LOW based on whether the amplified V_{HALL} exceeds a threshold.

- Small OpAmp offsets are absorbed by the comparator threshold — a lower-grade OpAmp is acceptable.
- The Schmitt trigger adds **hysteresis**: turn-on and turn-off thresholds differ slightly, preventing chattering near the boundary.

Key Takeaway — choosing a configuration:

- Need a *proportional* measurement of field strength? → Linear Hall sensor (OpAmp only). Requires a high-quality OpAmp; output is analog.
- Need only *presence/absence* detection? → Switch-type Hall sensor (OpAmp + comparator). Tolerates a lower-grade OpAmp; output is digital.

Slide 8 The SS49E Linear Hall Sensor

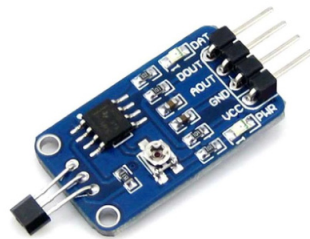


Figure 5: The SS49E linear Hall effect sensor module.

The SS49E is a specific **linear** Hall sensor — it uses Configuration 1 and provides a continuous analog output proportional to field strength. Key specifications:

- **Sensitivity:** $2.5 \pm 0.5 \text{ mV/G}$. For every Gauss of magnetic flux density, the output voltage changes by approximately 2.5 mV. The $\pm 0.5 \text{ mV/G}$ tolerance means unit-to-unit sensitivity variation of up to 20% — calibration is advisable in precision applications.
- **Power consumption:** **4 mA at 5 V DC**. Modest consumption; compatible with direct Arduino/microcontroller supply.

- **Minimum supply: 2.3 V.** The sensor can run from 3.3 V systems (Raspberry Pi, Nordic MCUs, etc.) without a level shifter.

Note — sensitivity tolerance. The $\pm 0.5 \text{ mV/G}$ spread means two sensors from the same batch may give readings that differ by up to 20% for the same field. For precise measurements, each unit should be individually calibrated against a known field.

Slide 9 Arduino Example — Reading the SS49E

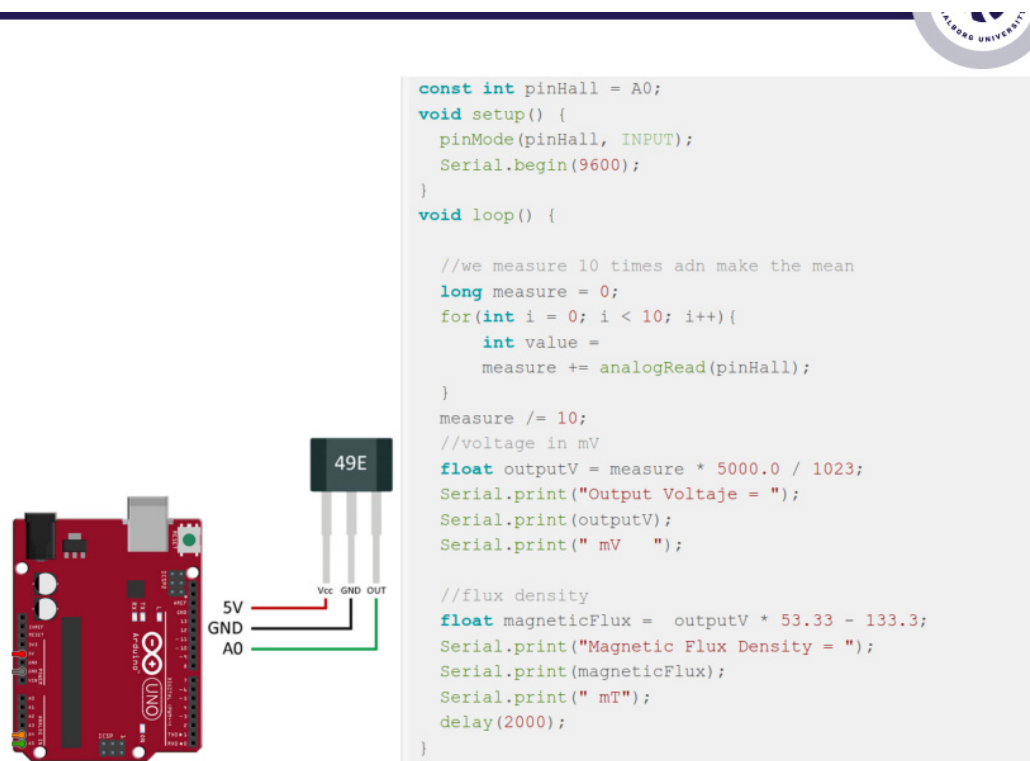


Figure 6: Wiring diagram and Arduino code for reading the SS49E.

The example code on slide 9 demonstrates a complete reading pipeline.

Step 1 — Averaging to reduce noise

```
long measure = 0;
for(int i = 0; i < 10; i++){
  measure += analogRead(pinHall);
}
measure /= 10;
```

Ten successive ADC readings are accumulated and divided by 10 — a basic **averaging filter**. Averaging N independent noise samples reduces the noise standard deviation by

a factor of $\sqrt{N}$, so averaging 10 readings gives $\sqrt{10} \approx 3.2\times$ noise reduction at the cost of 10 ADC conversions.

Step 2 — ADC count to millivolts

```
float outputV = measure * 5000.0 / 1023;
```

The Arduino Uno's ADC has **10-bit resolution**: it maps 0–5 V onto integers 0–1023. Multiplying by 5000/1023 recovers the voltage in millivolts.

Step 3 — Millivolts to magnetic flux density

```
float magneticFlux = outputV * 53.33 - 133.3;
```

This is the **inverse of the sensor's sensitivity curve**, converting millivolts back to flux density in millitesla. The subtraction of 133.3 accounts for the sensor's quiescent output at zero field (the output sits at mid-supply when no field is present, not at 0 V — this offset must be removed).

Key Takeaways — Hall sensor end-to-end:

- A Hall sensor converts **magnetic field strength** to a voltage via the Hall effect.
- V_{HALL} is tiny; an OpAmp is always needed.
- For field *strength* measurements use a linear sensor (analog output). For field *presence* detection use a switch-type sensor (digital output).
- The SS49E has sensitivity $2.5 \pm 0.5 \text{ mV/G}$; calibration is recommended for precision work.
- Averaging multiple ADC readings is a simple and effective way to reduce measurement noise — a preview of the filtering concepts covered later in this lecture.

Slides 10–11 Temperature Sensor: NTC Thermistor

A **Negative Temperature Coefficient (NTC) thermistor** is a resistor whose resistance *decreases* as temperature increases. It is one of the most common simple temperature sensors due to its low cost, wide availability, and large resistance change over temperature.

Voltage Divider Readout (Slide 11)

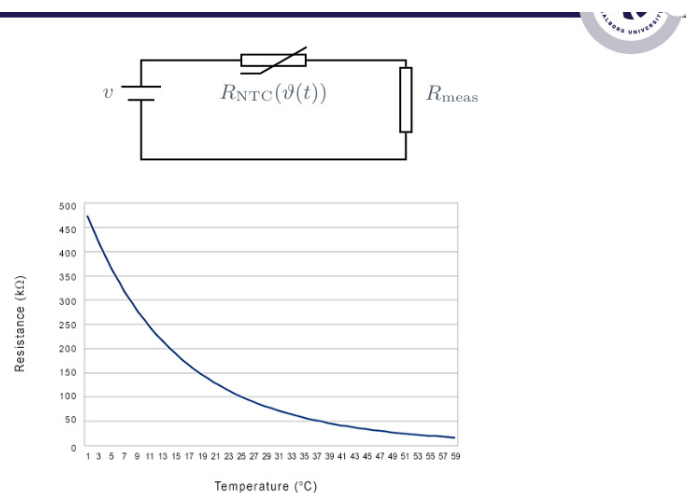
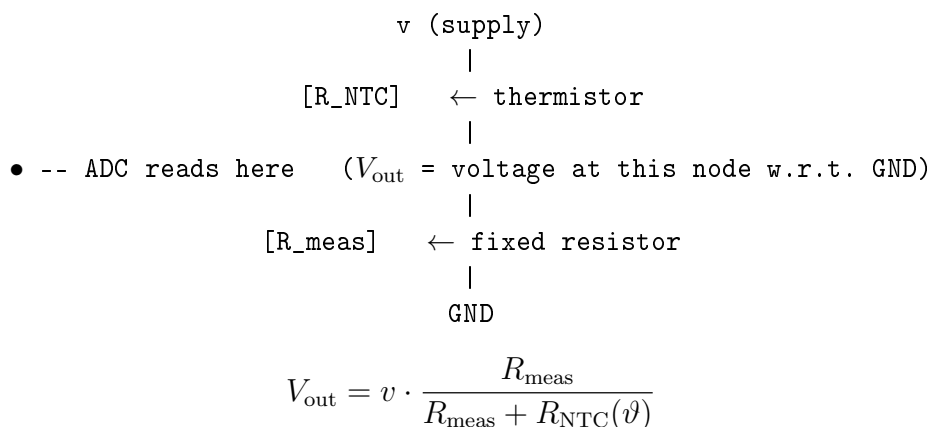


Figure 7: Voltage-divider circuit: v is the supply, R_{meas} is a fixed reference resistor, and $R_{NTC}(\vartheta(t))$ is the thermistor.

The NTC is read out using a simple **voltage divider**. A known supply voltage v is applied across the series combination of a fixed resistor R_{meas} and the thermistor R_{NTC} . The voltage *across* R_{meas} is fed to the ADC — “across” means the ADC is connected at the **node between the two resistors** (the positive end of R_{meas}) and at GND (its negative end). That node sits between the two components in the chain:



Because R_{NTC} changes with temperature ϑ , so does V_{out} — the ADC reading can then be converted back to temperature using the thermistor’s characteristic curve from its datasheet.

Note — choice of R_{meas} . Sensitivity is highest when $R_{\text{meas}} \approx R_{\text{NTC}}$ at the centre of your temperature range. Choosing R_{meas} equal to the thermistor's nominal resistance (e.g. $10\text{ k}\Omega$ at 25°C) is a common starting point.

The intuition comes from the two extremes:

- $R_{\text{meas}} \gg R_{\text{NTC}}$ — almost all of v drops across R_{meas} permanently; the thermistor barely influences V_{out} .
- $R_{\text{meas}} \ll R_{\text{NTC}}$ — almost all of v drops across R_{NTC} permanently; V_{out} sits near zero and barely moves.
- $R_{\text{meas}} = R_{\text{NTC}}$ — neither resistor dominates, so a small change in R_{NTC} shifts the balance the most, giving the largest swing in V_{out} .

Key Takeaway: An NTC thermistor requires only a fixed resistor and an ADC to read temperature. The non-linear R – T relationship means a lookup table or the Steinhart–Hart equation is needed to convert the ADC reading to a calibrated temperature.

Slides 12–14 Combined Sensor: DHT11

Sensor Overview (Slide 12)

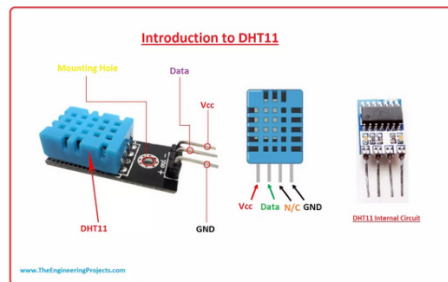


Figure 8: The DHT11 combined temperature and humidity sensor.

The **DHT11** integrates both a humidity sensor and a temperature sensor (NTC thermistor) in one package with a single-wire digital interface. Key specifications:

- **Humidity:** 20%–80 % relative humidity, accuracy $\pm 5\%$
- **Temperature:** 0°C to 50°C , accuracy $\pm 2^\circ\text{C}$
- **Supply:** 3.0 V to 5.5 V; current 0.3 mA (measuring), $60\text{ }\mu\text{A}$ (idle)
- **Sampling rate:** 1 Hz maximum
- **Response time:** 6 s to 15 s

Warning — slow response time. The DHT11 can only be sampled at 1 Hz, and its thermal and humidity response time is 6–15 s. It is unsuitable for applications that need fast or frequent measurements.

Sampling rate vs. response time:

- **Sampling rate** — how often you can *ask* for a reading (1 Hz).
- **Response time** — how long the physical element takes to *react* to a real change (6–15 s).

Even sampling at 1 Hz, a sudden jump in the environment is spread across multiple samples — the output creeps toward the true value rather than jumping in one step:

True humidity:	20%	-	-	-	-	-	80%
DHT11 output:	20%	25%	35%	50%	65%	75%	80%
Time:	0 s	1 s	2 s	3 s	4 s	5 s	6 s+

How the Humidity Sensor Works (Slide 13)

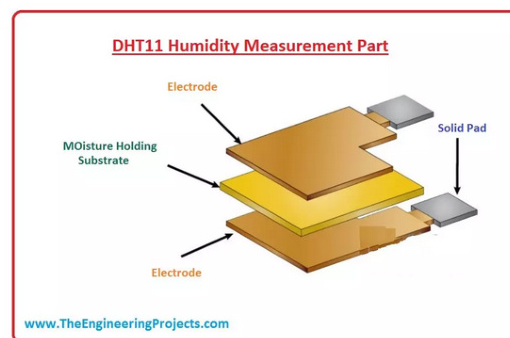


Figure 9: Capacitive humidity sensing principle inside the DHT11.

The humidity element is a **capacitive sensor** with two electrodes and a moisture-absorbing substrate in between:

1. The substrate material absorbs or releases water molecules as ambient humidity changes.
2. This alters the dielectric constant of the substrate, changing the **capacitance** between the two electrodes.
3. The on-chip circuit measures this capacitance change — analogous to how the NTC circuit measures resistance change — and converts it to a humidity reading.

Definition — Capacitive humidity sensor: a sensor that measures relative humidity by detecting the change in capacitance caused by moisture absorption in a dielectric substrate placed between two electrodes.

A **dielectric** is an insulating material that does not conduct electricity, but whose molecules polarise under an electric field — this polarisation is captured by the di-

electric constant ε . Capacitance between two electrodes is:

$$C = \varepsilon \cdot \frac{A}{d}$$

where A is the electrode area and d the distance between them. Both are fixed, so only ε changes — and water has a very high dielectric constant (≈ 80) vs. dry substrate ($\approx 2\text{--}5$), meaning even small moisture uptake causes a large, measurable shift in C :

High humidity $\rightarrow$ substrate absorbs water $\rightarrow \varepsilon \uparrow \rightarrow C \uparrow$

Low humidity $\rightarrow$ substrate releases water $\rightarrow \varepsilon \downarrow \rightarrow C \downarrow$

Communication Protocol (Slide 14)

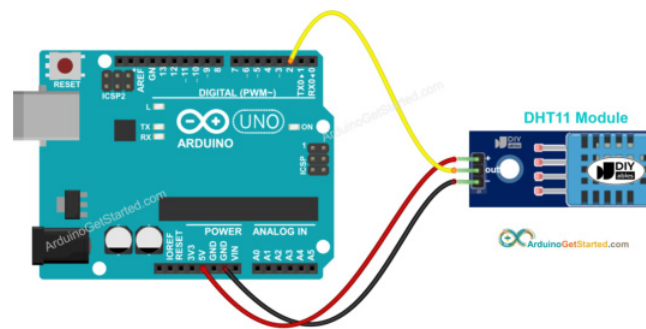


Figure 10: DHT11 single-wire protocol: activation pulse followed by 5 bytes of serial data.

The DHT11 uses a **single-wire serial protocol**:

1. **Activation:** the host pulls the data pin LOW for at least $18\mu\text{s}$ to wake the sensor.
2. **Data transmission:** the sensor responds with a 5-byte packet:
 - Byte 1: humidity integer part (8-bit, %)
 - Byte 2: humidity decimal part (always 0x00 for DHT11)
 - Byte 3: temperature integer part (8-bit, $^{\circ}\text{C}$)
 - Byte 4: temperature decimal part (always 0x00 for DHT11)
 - Byte 5: checksum (sum of bytes 1–4, lowest 8 bits)

Note — checksum verification. Always verify that $\text{Byte } 5 = (\text{Byte } 1 + \text{Byte } 2 + \text{Byte } 3 + \text{Byte } 4) \& 0\text{xFF}$ before using the reading. A mismatch means the transmission was corrupted and the data should be discarded and re-requested.

Key Takeaways — DHT11:

- Combines capacitive humidity sensing and NTC temperature sensing in one cheap

package with a single digital pin.

- Limited accuracy ($\pm 5\%$ RH, $\pm 2^\circ\text{C}$) and slow sampling (1 Hz, 6–15 s response) — suitable for non-critical environmental monitoring only.
- Always validate the checksum byte before trusting the reading.
- The decimal bytes (bytes 2 and 4) are always 0x00 for the DHT11 — not because of a protocol choice, but because the hardware lacks the accuracy to justify decimals ($\pm 2^\circ\text{C}$, $\pm 5\%$ RH). Reporting 23.4°C would be false precision. The DHT22 uses the same 5-byte protocol but fills those bytes with real fractional values ($\pm 0.5^\circ\text{C}$ accuracy) — the DHT11 simply leaves its precision slots empty.

Slide 15 Sensor Dynamics

Real sensors do not react instantaneously to changes in the physical world — they have **dynamics**: the sensor output takes time to settle to the true value after the input changes. This is captured mathematically using Newton’s law of cooling as the motivating example.

Newton’s Law of Cooling

“The rate of heat loss of a body is directly proportional to the difference in temperatures between the body and its surroundings, provided the temperature difference is small.”

Written as a differential equation:

$$\frac{d\vartheta(t)}{dt} = \frac{UA}{C_p}(\vartheta_a(t) - \vartheta(t))$$

where ϑ_a is the ambient temperature, UA is the product of surface area and heat transfer coefficient, and C_p is the sensor’s heat capacity.

First-Order State-Space Model

This is rewritten as a standard **first-order linear system**:

$$\dot{x}(t) = -\frac{UA}{C_p}x(t) + \frac{UA}{C_p}u(t), \quad x(0) = \vartheta(0)$$

$$y(t) = \beta x(t)$$

where $x(t)$ is the sensor’s internal temperature (state), $u(t)$ is the ambient temperature (input), and $y(t)$ is the sensor output scaled by a gain β .

Note — time constant. The speed of response is governed by the **time constant** $\tau = C_p/UA$. A sensor with large heat capacity C_p or poor thermal contact (small UA) responds slowly. This is exactly the same dynamic that makes the DHT11 need 6–15 s to respond to a humidity change.

Key Takeaway: Sensor dynamics mean the output $y(t)$ lags behind the true physical quantity. Modelling this as a first-order system lets you predict and compensate for the lag — important when the physical quantity changes faster than the sensor's time constant.

Slides 16–18 Measurement Noise and Sampling

Raw Data (Slide 16)

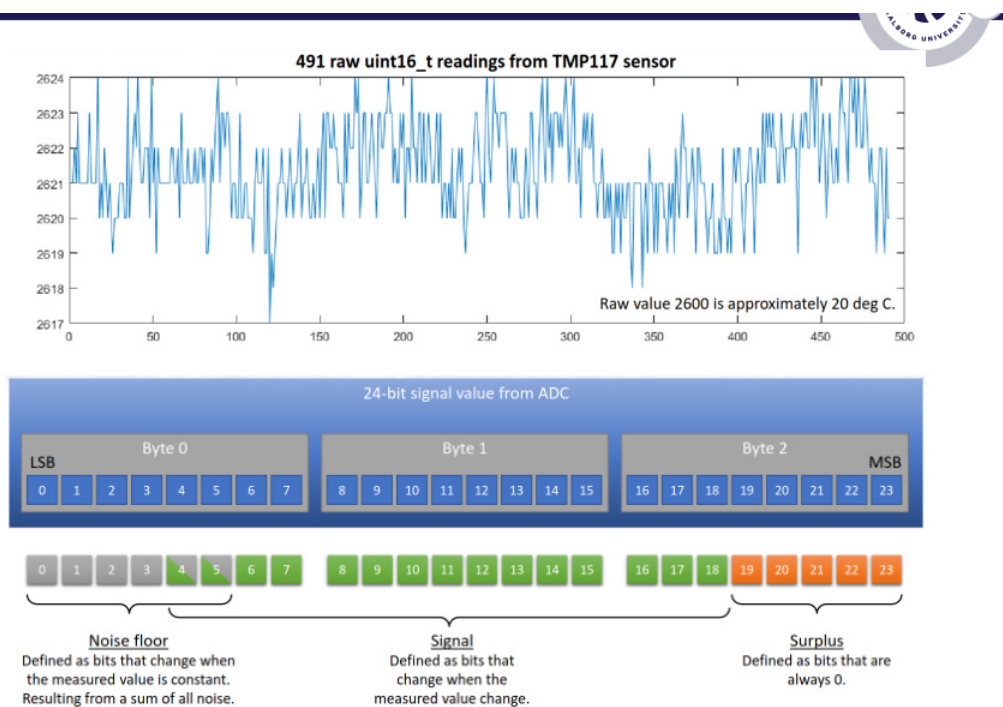


Figure 11: Raw sensor data showing the effect of measurement noise on a sampled signal.

Real sampled data never looks perfectly clean — noise from the sensor, the amplifier, and the ADC all contribute random fluctuations on top of the true signal. Slide 16 shows what this looks like in practice.

Noise as Part of the Sampling Process (Slide 17)

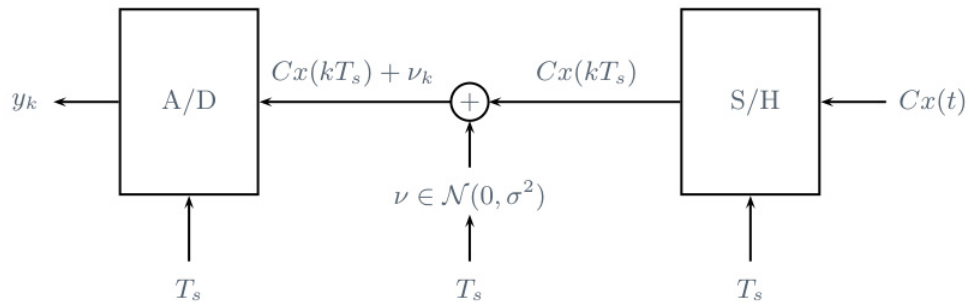


Figure 12: Block diagram: the noise ν_k is added at the sampling stage, after the sample-and-hold and ADC.

From a modelling perspective it is most convenient to treat measurement noise as part of the **sampling process** itself rather than as a property of the continuous-time signal. This means:

- The continuous-time output is $y(t) = Cx(t)$ (clean).
- After sampling, each discrete-time measurement picks up a noise sample ν_k :

$$y_k = Hx_k + Ju_k + \nu_k$$

- ν_k is drawn from a **Gaussian distribution** $\nu \sim \mathcal{N}(0, \sigma^2)$ — zero mean, variance σ^2 .

Note — why Gaussian? Each noise sample is the sum of many small, independent physical contributions (thermal agitation, quantisation rounding, EMI pick-up, etc.). By the Central Limit Theorem the sum of many independent random variables converges to a Gaussian regardless of the individual distributions.

Discrete-Time System with Noise (Slide 18)

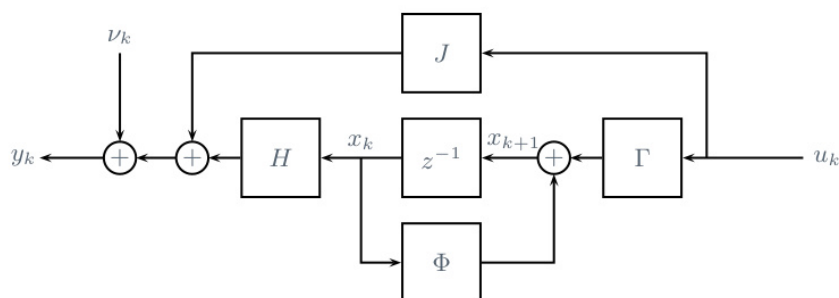


Figure 13: Discrete-time block diagram with measurement noise ν_k added at the output.

The full discrete-time model including measurement noise is:

$$x_{k+1} = \Phi x_k + \Gamma u_k \quad (\text{state equation})$$

$$y_k = Hx_k + Ju_k + \nu_k \quad (\text{output equation})$$

These two equations describe two different things and it is important to understand each separately:

State equation $x_{k+1} = \Phi x_k + \Gamma u_k$: This tells you how the *internal state* of the system (e.g. the true temperature inside the sensor) evolves from one time step to the next. Φ is how much of the current state carries forward, Γ is how much the input u_k (e.g. ambient temperature) drives the state. Crucially, **there is no noise here** — the true physical state evolves deterministically; we just cannot observe it directly.

Output equation $y_k = Hx_k + Ju_k + \nu_k$: This is what the sensor actually *reports*. H scales the state into a measurable quantity, J accounts for any direct feed-through from the input, and ν_k is the noise added at the moment of measurement. Every time you take a reading you get the truth *plus* a random kick from ν_k — which is why two readings of the same physical quantity will differ slightly.

The key insight is: the physical world is clean, the **act of measuring** is what introduces noise. We never have access to x_k directly — only to the corrupted y_k .

Key Takeaway: Measurement noise $\nu_k \sim \mathcal{N}(0, \sigma^2)$ only corrupts what we *observe* (y_k), not how the system actually evolves (x_k). The goal of a filter (e.g. Kalman filter) is to use the noisy y_k measurements to recover the best possible estimate of the clean x_k .

Slides 19–21 Probability, Normal Distribution, and Additive Noise

Basic Random Variable Statistics (Slide 19)

Probability Density Function (PDF) $f_X(x)$:

The PDF describes *how likely* the random variable X is to take values in a given region. It cannot be read as “the probability of exactly x ” (that is always zero for a continuous variable) — instead you **integrate it over an interval** to get a probability:

$$\Pr(a \leq X \leq b) = \int_a^b f_X(x) dx$$

Think of the PDF as a height map: tall regions are where values are more likely to land, flat regions are unlikely. The area under the entire curve always equals 1 (something must happen). Taking the integral between a and b gives you the **area** under the curve in that range, which equals the probability that a measurement falls between those two values.

Cumulative Distribution Function (CDF) $F_X(x)$:

The CDF answers a different question: “what is the probability that X is *at most* x ?”

$$F_X(x) = \Pr(X \leq x) = \int_{-\infty}^x f_X(u) du$$

You are integrating the PDF from $-\infty$ all the way up to x — accumulating all the probability to the left of x . This is why it starts at 0 (nothing is less than $-\infty$) and climbs to 1 (everything is less than $+\infty$). The PDF is simply the slope (derivative) of the CDF at each point:

$$f_X(x) = \frac{d}{dx} F_X(x)$$

Note — PDF vs CDF in plain terms:

- **PDF:** “how dense is the probability around this value?” — gives a shape, not a direct probability.
- **CDF:** “what fraction of outcomes fall below this value?” — directly readable as a probability between 0 and 1.

For noise: the CDF lets you answer “what is the chance a noise sample exceeds $\pm 3\sigma$?” without doing any integration yourself.

Two key summary statistics of any distribution:

$$\text{Mean: } \mu_X = E(X) = \int_{-\infty}^{\infty} x f_X(x) dx$$

Each possible value x is weighted by how likely it is ($f_X(x)$) and summed — this is the *long-run average* of many draws.

$$\text{Variance: } \sigma_X^2 = \text{Var}(X) = E[(X - \mu_X)^2] = E(X^2) - E(X)^2$$

$$\text{Standard deviation: } \sigma_X = \sqrt{\text{Var}(X)}$$

Standard deviation σ is the square root of the variance. It has the same units as X and tells you the typical size of a deviation from the mean. For a Gaussian: 68% of samples fall within $\pm\sigma$, 95% within $\pm 2\sigma$, 99.7% within $\pm 3\sigma$. For sensor noise this means: if $\sigma = 0.1^\circ\text{C}$, you can expect nearly all readings to be within $\pm 0.3^\circ\text{C}$ of the true value.

The Normal Distribution (Slide 20)

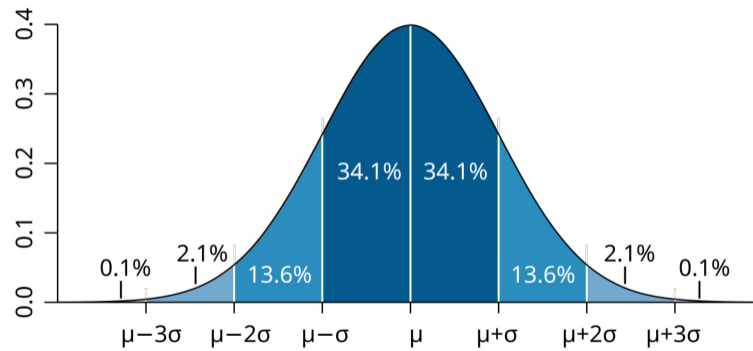


Figure 14: The Gaussian bell curve: PDF of $\mathcal{N}(\mu, \sigma^2)$.

The **normal (Gaussian) distribution** $X \sim \mathcal{N}(\mu, \sigma^2)$ has the PDF:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

μ sets the centre (mean) and σ^2 sets the width (spread). Measurement noise is modelled as $\mathcal{N}(0, \sigma^2)$ — zero mean because a systematic offset would be a calibration error, not noise.

Warning — σ vs σ^2 . The PDF is parameterised by σ^2 (variance), not σ . Confusing the two is a common mistake: $\mathcal{N}(0, 4)$ has standard deviation 2, not 4.

Additive Noise on a Sampled Signal (Slide 21)

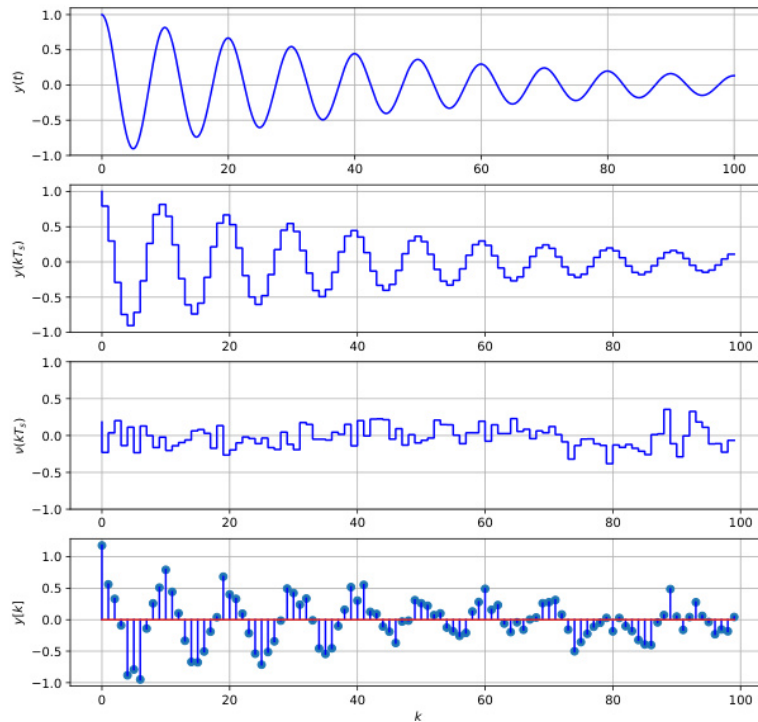


Figure 15: Top to bottom: (1) continuous nominal signal $y(t)$; (2) sampled signal $y(kT_s)$; (3) Gaussian noise $\nu(kT_s)$; (4) noisy sampled output $y[k] = y(kT_s) + \nu_k$.

Slide 21 visualises the full additive noise model in four panels:

1. **Nominal signal** $y(t)$ — the clean continuous-time signal the sensor ideally measures.
2. **Sampled signal** $y(kT_s)$ — the same signal held and sampled at intervals T_s by the S/H and ADC.
3. **Gaussian noise** $\nu(kT_s)$ — independent zero-mean noise drawn at each sample instant.
4. **Noisy output** $y[k]$ — the sum of panels 2 and 3; this is what the microcontroller actually receives.

Key Takeaways — noise and sampling:

- Measurement noise is modelled as additive Gaussian $\nu_k \sim \mathcal{N}(0, \sigma^2)$ at each sample.
- $\mu = 0$ by assumption — any non-zero mean is a bias/offset that should be removed by calibration.
- σ (standard deviation) is the key parameter describing noise magnitude; it has the same units as the measurement.
- The noisy output $y[k]$ is the only thing available to the algorithm — recovering the clean signal requires filtering.

Slides 22–23 Autocorrelation

What Is Autocorrelation? (Slide 22)

Step 1 — Cross-Correlation: comparing two different signals

Suppose you have **two signals** y_1 and y_2 and you want to know how *similar* they are — for example, is the output of one sensor driven by the same underlying phenomenon as another? **Cross-correlation** gives you a single number that answers this for every possible time shift between the two signals:

$$R_{y_1 y_2}(\kappa) = \sum_{k=-\infty}^{\infty} y_{1,k}^* y_{2,k+\kappa}$$

The result is a **high value** when the two signals are similar at that shift (they go up and down together), and a **low value** (near zero) when they are unrelated. A **negative value** means they move in opposite directions at that shift (one rises while the other falls).

Step 2 — Lag: comparing at different time offsets

The argument κ is called the **lag**. Rather than asking “are these two signals similar right now?” you are asking “are they similar when one is shifted by κ samples relative to the other?”

Think of it this way: imagine sliding one signal along the time axis while the other stays fixed, and at each position computing how much they overlap. Each position corresponds to one lag value κ :

$\kappa = 0$	compare $y_1[k]$ against $y_2[k]$ -- same time instant
$\kappa = 1$	compare $y_1[k]$ against $y_2[k+1]$ -- y_2 shifted 1 sample ahead
$\kappa = -1$	compare $y_1[k]$ against $y_2[k-1]$ -- y_2 shifted 1 sample behind

A large peak at lag $\kappa = 3$, for example, means: “ y_2 looks a lot like y_1 , but delayed by 3 samples.” This is extremely useful for finding time delays between two sensor readings of the same event.

Step 3 — Autocorrelation: comparing a signal with itself

Autocorrelation is the special case where you cross-correlate a signal *with itself* — $y_1 = y_2 = y$:

$$R_y(\kappa) = \sum_{k=-\infty}^{\infty} y_k^* y_{k+\kappa}$$

You are now asking: “does this signal resemble a shifted version of itself?” In other words, **does the past predict the future?**

Note — what the autocorrelation value tells you.

- **High** $R_y(\kappa)$ at some lag $\kappa \neq 0$: the signal is correlated with its own past —

knowing a value κ steps ago tells you something useful about the current value. The signal has *memory* and is predictable.

- **Near zero** $R_y(\kappa)$ for all $\kappa \neq 0$: each sample is independent of every other — the signal has *no memory*. This is the signature of white noise.
- **At lag** $\kappa = 0$: comparing the signal to itself with no shift always gives the maximum — equal to the total energy of the signal.

White Noise vs Periodic Signal (Slide 23)

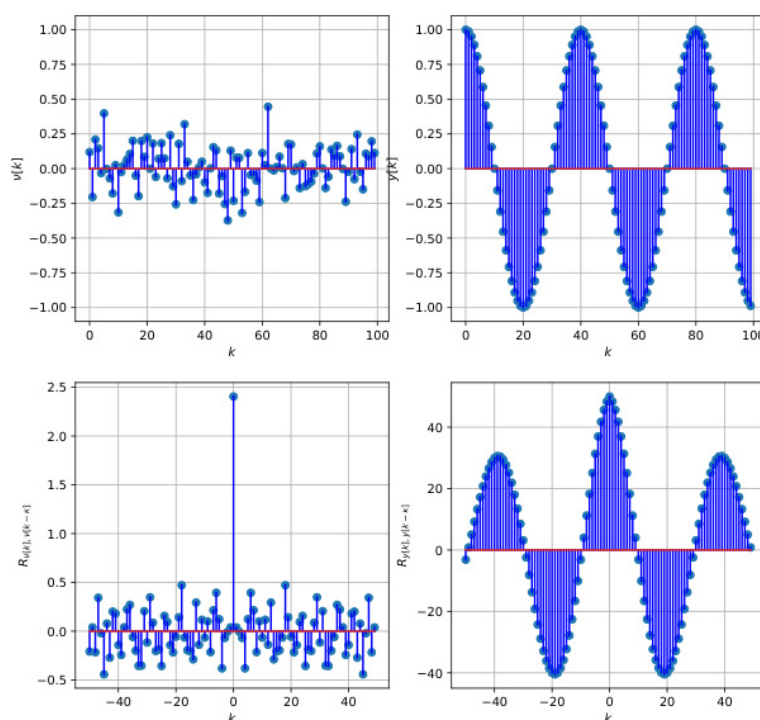


Figure 16: Left column: white noise $\xi[k]$ and a periodic signal $y[k]$. Right column: their autocorrelations R_{ξ} and R_y .

The plots show the autocorrelation for two very different signals:

- **White noise** (top row): the autocorrelation is a sharp spike at $\kappa = 0$ and essentially zero everywhere else. This means each sample is completely independent of every other — knowing one sample tells you nothing about the next.
- **Periodic signal** (bottom row): the autocorrelation is itself periodic, with peaks repeating at the same period as the signal. This makes sense — shift a sine wave by exactly one period and it looks identical to itself.

Key Takeaway: Autocorrelation is a fingerprint of a signal's structure. A sharp spike at zero $\rightarrow$ no memory (noise). Repeating peaks $\rightarrow$ periodic structure. This lets you detect hidden periodicity in noisy data even when it is invisible in the raw time series.

Slide 24 Power Spectral Density

Power Spectral Density (PSD) answers the question: **how is the signal's energy (or power) distributed across frequencies?**

Parseval's Theorem

Parseval's theorem states that the total energy of a signal is the same whether computed in the time domain or the frequency domain:

$$E = \int_{-\infty}^{\infty} |y(t)|^2 dt = \int_{-\infty}^{\infty} |Y(\omega)|^2 d\omega$$

where $Y(\omega)$ is the Fourier transform of $y(t)$. This means $|Y(\omega)|^2$ tells you how much energy sits at each frequency ω .

Energy Spectral Density

The **energy spectral density** is defined as:

$$\bar{S}_{yy}(\omega) = |Y(\omega)|^2$$

The value $\bar{S}_{yy}(\omega) d\omega$ is the energy contained in the infinitesimally small frequency band $[\omega, \omega + d\omega]$. Integrating over all frequencies recovers the total energy — consistent with Parseval.

Note — PSD and autocorrelation are a Fourier pair. The Wiener–Khinchin theorem states that the PSD $\bar{S}_{yy}(\omega)$ and the autocorrelation $R_y(\tau)$ are a Fourier transform pair:

$$\bar{S}_{yy}(\omega) = \mathcal{F}\{R_y(\tau)\}$$

This links the two concepts directly: a signal with no memory (flat autocorrelation spike) has a *flat* PSD — equal energy at all frequencies. That is precisely the definition of white noise.

Key Takeaway: The PSD is the frequency-domain view of how a signal's energy is spread. A flat PSD means equal power at all frequencies (white noise). A PSD that falls off at high frequencies means the signal is smoother and more correlated in time.

Slide 25 Coloured Noise

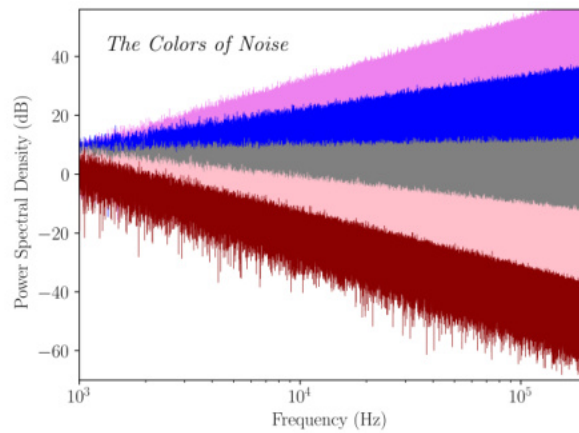


Figure 17: Noise colours and their power spectral density shapes.

Not all noise is white. **Coloured noise** refers to noise whose PSD is *not* flat — power is concentrated at certain frequencies. The colours are an analogy to visible light spectra:

- **White noise** — equal power at all frequencies. Each sample is fully independent of every other (zero autocorrelation for $\kappa \neq 0$). The baseline reference.
- **Grey noise** — similar to white but shaped to match the sensitivity of the human ear (perceptually flat).
- **Pink noise ($1/f$ noise)** — power decreases as frequency increases ($S \propto 1/f$). Sounds deeper than white noise. Common in electronics (flicker noise in transistors).
- **Red / Brown noise** — even steeper roll-off at high frequencies ($S \propto 1/f^2$). Sounds like thunder or distant rumble. Result of integrating white noise (Brownian motion).
- **Blue noise** — power increases with frequency. Used in dithering and audio applications.
- **Violet noise** — steepest high-frequency emphasis ($S \propto f^2$). Sounds like a running faucet.

Warning — assuming white noise. Most simple sensor models assume ν_k is white Gaussian noise. In practice, real sensor noise often has a coloured component (e.g. pink flicker noise at low frequencies). If the noise is coloured, a filter designed for white noise will be suboptimal — the noise model should be validated against measured data.

Key Takeaway: Noise colour describes the shape of the PSD. White = flat (no memory). Colours with more low-frequency power (pink, red) mean adjacent samples are correlated — the noise has memory. This directly connects back to autocorrelation: coloured noise has a non-trivial $R_y(\kappa)$ for $\kappa \neq 0$.



Figure 18: Filter block $F(\cdot)$ taking noisy signal y_k and producing filtered output $\bar{y}_k$.

A **discrete filter** $F(\cdot)$ takes the noisy measurement y_k and produces a cleaned-up version $\bar{y}_k$. The signal y_k has two components:

$$y_k = y_{d,k} + \nu_k$$

where $y_{d,k}$ is the **deterministic** part we want to keep, and ν_k is the noise we want to remove. After filtering:

$$\bar{y}_k = \bar{y}_{d,k} + \epsilon_k = F(y_{d,k}) + F(\nu_k)$$

The goal of filtering is to simultaneously achieve:

- $\bar{y}_{d,k} \approx y_{d,k}$ — preserve the true signal as faithfully as possible.
- $|\epsilon_k| \ll |\nu_k|$ — shrink the noise as much as possible.

These two goals are in tension: an aggressive filter kills noise but also distorts or delays the signal. Every filter design is a trade-off between the two.

Note — three implementation approaches.

- **Impulse response / convolution** — directly apply the filter's impulse response via convolution with the signal. Intuitive but computationally expensive for long filters.
- **Transfer function / difference equation** — express the filter as a ratio of polynomials in z and convert to a recursive difference equation. Efficient and common in practice.
- **State space** — represent the filter as a first-order vector/matrix system. Most flexible; the same form used for the physical system model, making it easy to combine sensor and filter in one framework.

Slides 27–30 The Averaging Filter

Impulse Response and Convolution (Slide 27)

The simplest filter for reducing high-frequency noise is the **averaging filter**: take the last m samples and output their mean. Its impulse response is:

$$f_{\text{av},k} = \left\{ \frac{1}{m}, \frac{1}{m}, \dots, \frac{1}{m}, 0, 0, \dots \right\} \quad (m \text{ non-zero terms})$$

Applied via discrete-time convolution:

$$\bar{y}_k = f_{\text{av},k} * y_k = \sum_{\kappa=-\infty}^{\infty} f_{\text{av},\kappa} y_{k-\kappa} = \frac{1}{m} \sum_{\kappa=0}^{m-1} y_{k-m+1+\kappa}$$

This is simply the running mean of the last m samples.

Why does averaging remove noise?

Recall that noise is modelled as $\nu_k \sim \mathcal{N}(0, \sigma^2)$ — **zero mean**. This is the key. Each individual noise sample is a random kick up or down, but on average they cancel.

When you average m samples of the noisy signal:

$$\bar{y}_k = \frac{1}{m} \sum_{\kappa=0}^{m-1} y_{k-\kappa} = \frac{1}{m} \sum_{\kappa=0}^{m-1} y_{d,k-\kappa} + \underbrace{\frac{1}{m} \sum_{\kappa=0}^{m-1} \nu_{k-\kappa}}_{\text{averaged noise}}$$

The averaged noise term has variance σ^2/m — it shrinks by a factor of m , so the standard deviation shrinks by $\sqrt{m}$. More samples $\rightarrow$ the random positive and negative kicks cancel each other out more completely $\rightarrow$ the average gets closer to the true signal. This is the **law of large numbers** at work: the sample mean of zero-mean noise converges to zero as $m \rightarrow \infty$.

Note — the $\sqrt{m}$ rule. Doubling m does not halve the noise — it reduces it by a factor of $\sqrt{2} \approx 1.4$. To get $10\times$ noise reduction you need $100\times$ more samples. This diminishing return is why dedicated filter designs (Bessel, Butterworth) are preferred when strong noise rejection is required.

Why convolution and not just sum-and-divide?

For the averaging filter, **they are the same thing** — the convolution formula simply writes the sum-and-divide in a general way. The reason to express it as convolution is that it generalises:

- **Averaging** assigns equal weight $1/m$ to each of the last m samples. The impulse response is a flat rectangle.

- A **general filter** can assign *any* weights to any number of past samples. The impulse response is that set of weights.
- Convolution is the single operation that covers all cases — slide the weight vector across the signal, multiply element-wise, sum. Swap in a different impulse response and you get a completely different filter using the same code.

So convolution is not a more complex way of averaging — it is the **universal language** for describing what any linear filter does to a signal, of which averaging is just the simplest special case.

Convolution in Code (Slide 28)

Both Python and C implementations follow the same nested-loop structure: the outer loop steps through each output sample k , the inner loop accumulates the weighted sum of past inputs.

Python

```
def conv(f, y):
    N = len(y)
    n = len(f)
    filtered_y = np.zeros(N + n)
    for k in range(N + n):
        for i in range(n):
            if (k - i >= 0) and (k - i < N): # implicit zero-padding
                filtered_y[k] += f[i] * y[k - i]
    return filtered_y
```

// C

```
int conv(double *f, double *y, int N, int n, float *filtered_y) {
    for (int k = 0; k < N + n; k++) {
        for (int i = 0; i < n; i++) {
            if ((k - i >= 0) && (k - i < N))
                filtered_y[k] += f[i] * y[k - i];
        }
    }
    return 0;
}
```

Warning — output length. The output of convolving an N -sample signal with an n -tap filter has $N + n - 1$ samples, not N . The extra samples at the start are the filter “warming up” with zero-padded input. Always allocate $N + n$ elements for the output array in C.

Impulse and Frequency Response (Slide 29)

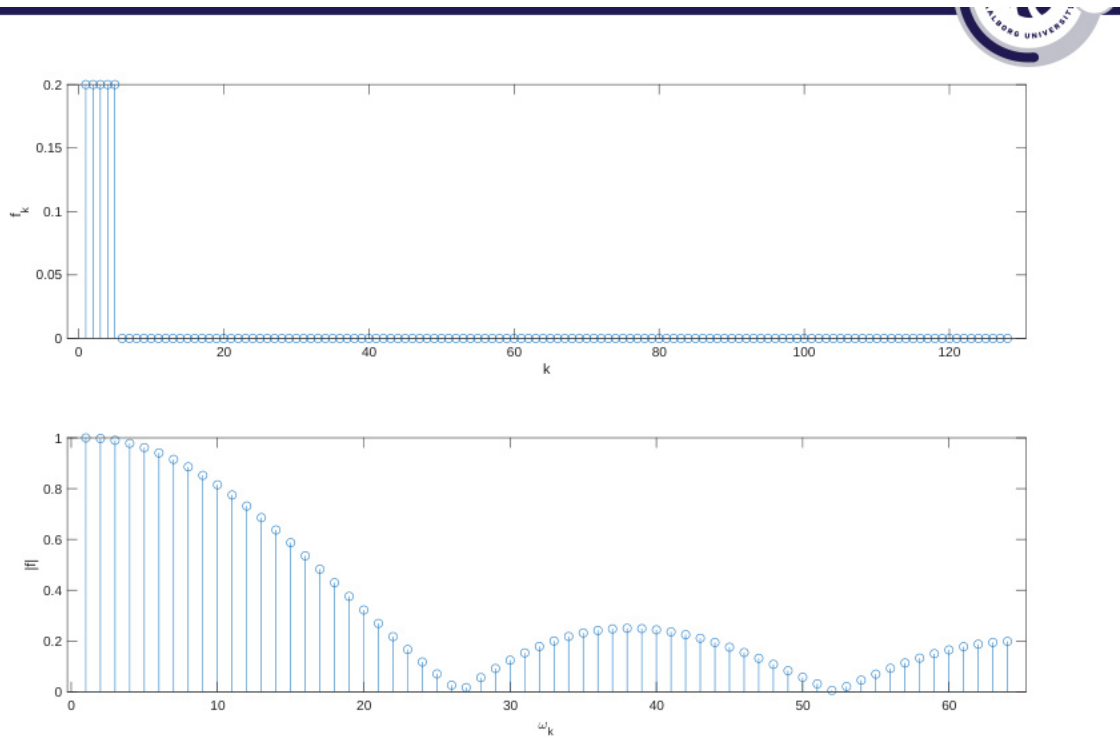


Figure 19: Left: impulse response f_k of the averaging filter (box of height $1/m$). Right: frequency response magnitude $|F|$ showing the low-pass characteristic.

The impulse response (left plot) is a rectangular window of height $1/m$ and width m samples — flat within the window, zero outside.

The frequency response (right plot) shows the **low-pass** characteristic: low frequencies pass with gain ≈ 1 , high frequencies are attenuated. However, the attenuation is gradual and the filter has **sidelobes** (the ripple at higher frequencies), meaning it does not cut off sharply. This is the main limitation of the averaging filter compared to purpose-designed filters.

Bode Plot (Slide 30)

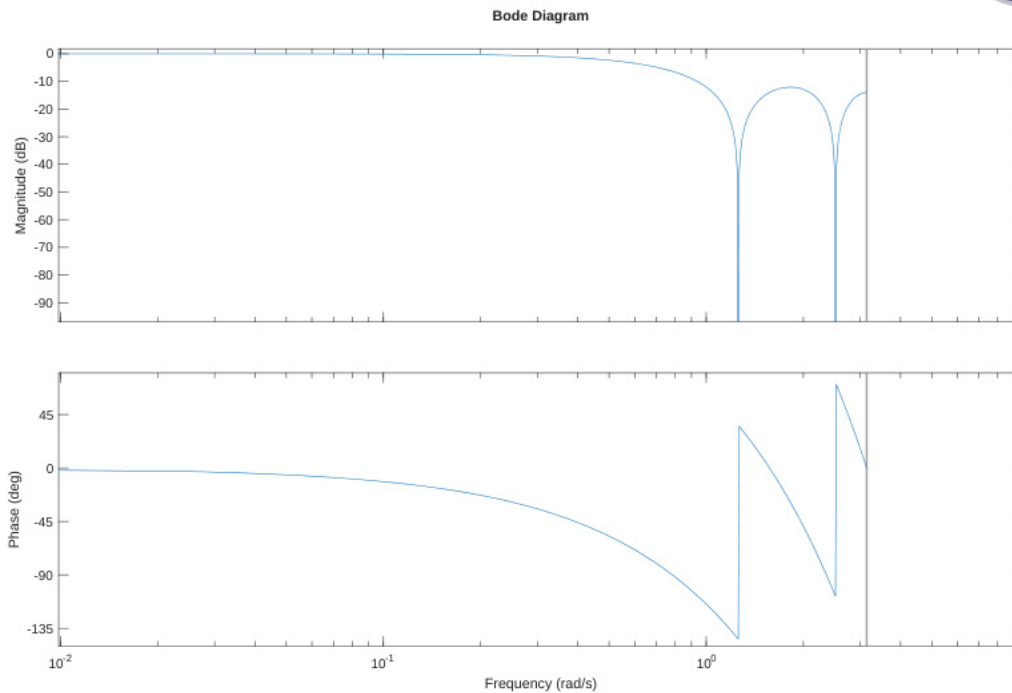


Figure 20: Bode plot of the averaging filter: magnitude (top) and phase (bottom) vs. frequency.

The Bode plot shows both magnitude and phase response on a logarithmic frequency axis. Key observations:

- **Magnitude:** rolls off at higher frequencies, confirming the low-pass behaviour. The roll-off is not steep — the averaging filter is a weak low-pass.
- **Phase:** introduces a frequency-dependent phase shift (delay). A constant-phase filter would delay all frequencies equally; the averaging filter does not, which means it distorts the shape of fast-changing signals.

Key Takeaways — averaging filter:

- Simple to implement; just a running mean of the last m samples.
- Works by convolution with a rectangular impulse response.
- Low-pass behaviour: smooths out high-frequency noise.
- Limitations: gradual roll-off, sidelobes, and phase distortion — better filters exist for demanding applications.
- Larger m = more smoothing but also more **lag**: because the output at time k is the mean of the last m samples, it is centred around $k - m/2$ — the filter is always looking $m/2$ steps into the past. A real change in the signal only fully shows up

in the output after m new samples have been collected. More smoothing = older samples included = the output takes longer to catch up with reality.

Slides 31–32 Bessel and Butterworth Filters

Filter Designs (Slide 31)

Purpose-designed filters offer much better performance than the averaging filter. The two most common choices for sensor applications are:

Bessel filter:

- Designed for **maximally linear phase response** — $d\varphi(\omega)/d\omega$ is as constant as possible.
- A constant phase slope means all frequencies are delayed by the same amount, so the **wave shape is preserved** in the passband. No distortion of the signal's time-domain structure.
- Harder to design directly in discrete time — typically designed in continuous time and then discretised.

Butterworth filter:

- Designed for **maximally flat magnitude** in the passband — the gain is as close to 1 as possible across all passband frequencies.
- Steeper roll-off than Bessel at the cost of some phase non-linearity.
- Can be discretised directly using Tustin's method (bilinear transform), making it more straightforward to implement.

Note — Bessel vs Butterworth trade-off.

- Need to preserve signal *shape* (e.g. a step response or pulse)? → Bessel (linear phase).
- Need a flat passband and sharp roll-off? → Butterworth (flat magnitude).

Neither is universally better; the choice depends on whether timing or amplitude accuracy matters more.

Frequency Responses Compared (Slide 32)

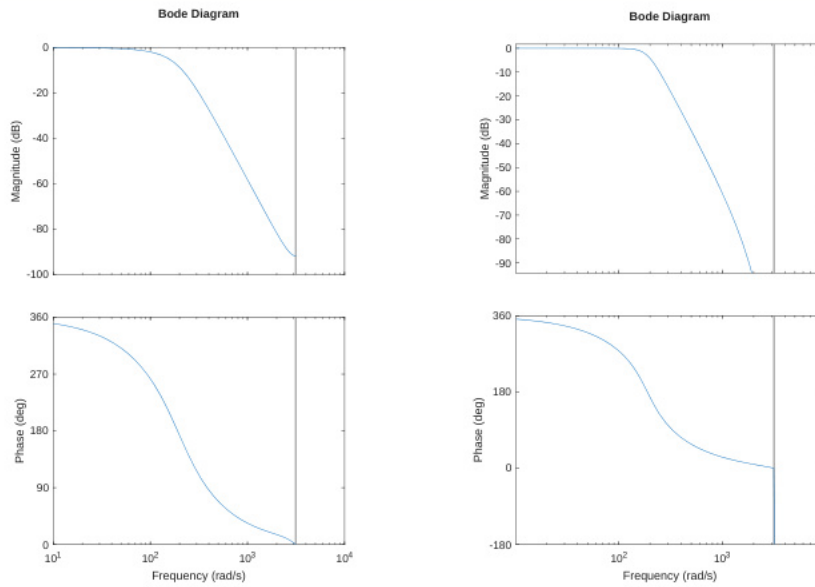


Figure 21: Bode plots at cut-off 30 Hz, sampling 1 kHz. Left: Bessel. Right: Butterworth.

At the same cut-off frequency (30 Hz) and sampling rate (1 kHz) the difference is visible in the Bode plots: the Butterworth has a flatter passband and steeper roll-off, while the Bessel has a more gradual roll-off but better-behaved phase.

The discrete-time transfer functions are:

$$F_{\text{Bes}}(z) = \frac{10^{-3}(0.1814z^3 + 0.6253z^2 + 0.1351z)}{z^4 - 3.43z^3 + 4.431z^2 - 2.556z + 0.555}$$

$$F_{\text{But}}(z) = \frac{10^{-3}(0.06239z^4 + 0.2495z^3 + 0.3743z^2 + 0.2495z + 0.06239)}{z^4 - 3.508z^3 + 4.641z^2 - 2.743z + 0.6105}$$

Slides 33–34 Filter Implementation

Difference Equation (Slide 33)

$$\bar{y}_k = \frac{b_m z^m + b_{m-1} z^{m-1} + \dots + b_1 z + b_0}{z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0} y_k \Rightarrow$$

$$\bar{y}_k = b_m y_k + b_{m-1} y_{k-1} + \dots + b_1 y_{k-m+1} + b_0 y_{k-m}$$

$$- a_{n-1} \bar{y}_{k-1} - \dots - a_1 \bar{y}_{k-n+1} - a_0 \bar{y}_{k-n}$$

Figure 22: Transfer function converted to a difference equation (numerator coefficients b , denominator a).

Any filter expressed as a rational z -domain transfer function:

$$\frac{\bar{Y}(z)}{Y(z)} = \frac{b_m z^m + b_{m-1} z^{m-1} + \dots + b_0}{z^n + a_{n-1} z^{n-1} + \dots + a_0}$$

can be converted into a **difference equation** that is computed recursively sample by sample:

$$\bar{y}_k = b_m y_k + b_{m-1} y_{k-1} + \dots + b_0 y_{k-m} - a_{n-1} \bar{y}_{k-1} - \dots - a_0 \bar{y}_{k-n}$$

Each new output $\bar{y}_k$ depends only on the current and past inputs y and past outputs $\bar{y}$ — no convolution sum required, just a short weighted sum updated every sample.

```
def filter(b, a, y):
    yfilt = np.zeros_like(y)
    for k in range(len(y)):
        s = 0.0
        for i in range(len(b)):
            if k - i > -1:
                s += b[i] * y[k - i]
        for i in range(1, len(a)):
            if k - i > -1:
                s -= a[i] * yfilt[k - i]
        yfilt[k] = s / a[0]
    return yfilt
```

Note — IIR vs FIR. A filter whose difference equation includes past *output* terms ($\bar{y}_{k-i}$) is called **IIR** (Infinite Impulse Response) — feedback makes its impulse response potentially infinite. The averaging filter has no feedback and is **FIR** (Finite Impulse Response). Bessel and Butterworth are IIR.

State Space Formulation (Slide 34)



$$x_{k+1} = \Phi x_k + \Gamma y_k$$

$$\bar{y}_k = H x_k + J y_k$$

where

$$\Phi = \begin{bmatrix} -a_n & -a_{n-1} & \dots & -a_1 & -a_0 \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{bmatrix}, \quad \Gamma = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix},$$

$$H = [b_{n-1} - b_n a_{n-1} \quad b_{n-2} - b_n a_{n-2} \quad \dots \quad b_1 - b_n a_1 \quad b_0 - b_n a_0], \quad J = b_n$$

and $b_n = b_{n-1} = \dots = b_{m+1} = 0$ whenever $m < n$.

Figure 23: State space matrices Φ , Γ , H , J for the discrete-time filter.

The same filter can be written in **state space form**:

$$x_{k+1} = \Phi x_k + \Gamma y_k$$

$$\bar{y}_k = H x_k + J y_k$$

This is structurally identical to the sensor model from slide 18, see Figure — the filter is itself a discrete-time dynamical system, with y_k (the noisy measurement) as its input and $\bar{y}_k$ (the filtered output) as its output. The matrices Φ , Γ , H , J are populated directly from the a and b coefficients of the transfer function.

Key Takeaway — three equivalent representations:

- **Convolution** — intuitive, expensive for long filters.
- **Difference equation** — efficient, easy to code sample-by-sample.
- **State space** — most powerful; unifies filter and system model, enables Kalman filtering. Use linear algebra libraries in practice.

Slide 35 Filtering Performance

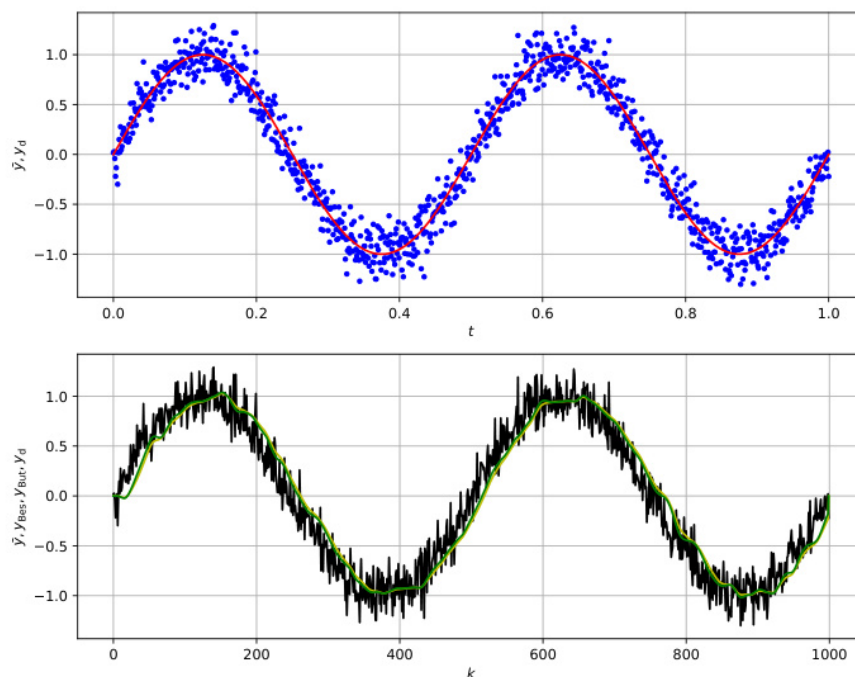


Figure 24: Top: continuous-time signal. Bottom: noisy discrete samples with Bessel (yellow) and Butterworth (green) filtered outputs vs. true deterministic signal (red).

The performance plot compares the two filters on a noisy sampled signal:

- **Red** — the true deterministic signal y_d (what we want to recover).
- **Yellow** — Bessel filtered output. Follows the shape of the true signal faithfully with minimal phase distortion, but leaves slightly more noise through due to its gradual roll-off.
- **Green** — Butterworth filtered output. Removes more noise (steeper roll-off) but introduces slightly more phase lag and shape distortion, visible at sharp transitions.

Key Takeaway — no free lunch. Both filters reduce noise significantly compared to the raw signal. Butterworth suppresses more noise; Bessel preserves shape better. The “right” choice depends on whether the application is more sensitive to noise amplitude or to signal timing and shape.

Slide 36 Lecture Summary

The lecture covered three broad areas:

Simple sensors: A sensor is “simple” if it measures one continuous physical quantity, provides a straightforward analog or digital interface, and requires no extensive processing. Examples covered: Hall effect (magnetic field), NTC thermistor (temperature), DHT11 (temperature + humidity).

Noise:

- **Gaussian / white noise** — normally distributed, equal power at all frequencies, zero autocorrelation for $\kappa \neq 0$. The standard model for measurement noise.
- **Coloured noise** — unequal power across frequencies, but still uncorrelated with time-shifted versions of itself. Pink and red noise have more low-frequency power; blue and violet have more high-frequency power.

Filtering — why low-pass filters remove noise: The key insight is in the frequency domain. The true signal $y_{d,k}$ is typically **slowly varying** — its energy is concentrated at *low* frequencies. Measurement noise ν_k is i.i.d. (independent and identically distributed), which means each sample is independent of every other — there is no correlation between adjacent samples. This is exactly the definition of **white noise**: flat power spectral density, equal energy at *all* frequencies, including the high ones.

A **low-pass filter** passes low frequencies and attenuates high frequencies. Since the signal lives at low frequencies and the noise is spread across all frequencies, the filter preferentially keeps the signal and cuts the noise:

Component	Frequency content	After low-pass
True signal $y_{d,k}$	concentrated at low f	passes through
i.i.d. noise ν_k	spread across all f equally	high- f portion cut

The more i.i.d. (white) the noise, the more evenly spread it is, and the more of it a low-pass filter removes. This is why assuming white Gaussian noise is both convenient and practically effective.

Bessel and Butterworth are both low-pass filters with a sharper, better-controlled cut-off than the averaging filter — Butterworth maximises how flat the passband is (signal distortion minimised), Bessel maximises phase linearity (shape of the signal preserved). Three implementations (convolution, difference equation, state space) are equivalent descriptions of the same filter.

Overall Key Takeaway: Every real sensor measurement is $y_k = y_{d,k} + \nu_k$. The sensor determines $y_{d,k}$; the noise model describes ν_k ; the filter recovers the best estimate of $y_{d,k}$ from the corrupted y_k . These three elements — sensor, noise model, filter — form the complete picture of a simple sensor system.